

DESIGN AND ANALYSIS OF ALGORITHMS

Question Bank Solutions

Modules I and II — All Questions Answered in Detail

Module I — Introduction, Divide & Conquer

Module II — Disjoint Sets, Backtracking

MODULE I — INTRODUCTION TO ALGORITHMS, DIVIDE AND CONQUER

Short Answer Questions

Q1. Define an Algorithm.

An **algorithm** is a finite sequence of well-defined, unambiguous instructions used to solve a particular problem. Each instruction has a clear meaning and can be performed with a finite amount of effort in a finite length of time. An algorithm takes an input, processes it step by step, and produces the required output. It is independent of any programming language and can be written in plain language, pseudocode, or a flowchart before being converted into an actual program.

Q2. Explain the characteristics of an Algorithm.

The essential characteristics of an algorithm are:

- **Input:** there are zero or more quantities, which are externally supplied to the algorithm.
- **Output:** at least one quantity is produced as the result of the algorithm.
- **Definiteness:** each instruction must be clear and unambiguous — every step has only one meaning.
- **Finiteness:** if the instructions are traced, the algorithm must terminate after a finite number of steps for all cases.
- **Effectiveness:** every instruction must be sufficiently basic that it can, in principle, be carried out by a person using only pencil and paper.

Q3. What is the difference between Time Complexity and Space Complexity?

Time complexity measures how much time (number of basic operations) an algorithm takes to run, expressed as a function of the input size n . It tells us how the running time grows as the input grows.

Space complexity measures how much memory an algorithm requires during its execution, also as a function of the input size — it includes the fixed part (instructions, constants, simple variables) plus the variable part (arrays, dynamic memory, recursion stack). Both help evaluate and compare the efficiency of different algorithms for the same problem.

Q4. What is the Divide and Conquer Technique?

Divide and Conquer is an algorithm design technique in which a problem is repeatedly divided into smaller subproblems of the same type, each subproblem is solved independently (usually recursively), and finally their solutions are combined to obtain the solution of the original problem. It has three steps — **Divide** the problem into subproblems, **Conquer** each subproblem recursively, and **Combine** the sub-solutions. Common examples are Binary Search, Merge Sort, Quick Sort, and Strassen's Matrix Multiplication.

Q5. How many multiplications are required in Strassen's algorithm?

In conventional multiplication of two 2×2 matrices, **8 multiplications** are required. Strassen's Matrix Multiplication reduces the number of multiplications to **7**, at the cost of more additions and subtractions (**18** additions/subtractions). For $n \times n$ matrices, this reduces the time complexity from $O(n^3)$ to $O(n^{\log_2 7}) \approx O(n^{2.807})$.

Q6. What is a pivot element in Quick Sort?

A pivot element is the element selected from the array during the Quick Sort process to partition the array into two parts. Elements smaller than the pivot are placed on its left side, and elements greater than the pivot are placed on its right side. After partitioning, the pivot is placed in its correct sorted position, and the two partitions are sorted recursively. Any element can be chosen as the pivot — the first, last, middle, or a random element.

Long Answer Questions

Q1. Explain the Tower of Hanoi problem in detail. Write the recursive algorithm for solving it and describe the steps involved in moving the disks from source to destination.

The **Tower of Hanoi** is a classic mathematical puzzle that beautifully demonstrates the concept of **recursion**. It consists of three rods (pegs): **Source**, **Auxiliary** and **Destination**, and a number of disks of different sizes placed on the Source rod in ascending order of size (smallest at the top, largest at the bottom). The objective is to move the entire stack of disks from the Source rod to the Destination rod using the Auxiliary rod, without ever changing the order of the disks.

Rules of the problem

- Only one disk can be moved at a time.
- Only the top disk of any rod can be moved.
- A larger disk can never be placed on top of a smaller disk.

Objective: move all disks from Source to Destination using the Auxiliary rod, preserving the order.

Steps for 3 disks

- Move disk 1 from Source → Destination
- Move disk 2 from Source → Auxiliary
- Move disk 1 from Destination → Auxiliary
- Move disk 3 from Source → Destination
- Move disk 1 from Auxiliary → Source
- Move disk 2 from Auxiliary → Destination
- Move disk 1 from Source → Destination

Total moves required = $2^3 - 1 = 7$ moves.

Recursive algorithm

```
Algorithm TOH(n, Source, Auxiliary, Destination) { if (n >= 1) then { TOH(n - 1, Source, Destination, Auxiliary) write("Move disk ", n, " from ", Source, " to ", Destination) TOH(n - 1, Auxiliary, Source, Destination) } }
```

Parameters: $n \rightarrow$ number of disks; Source $\rightarrow$ the rod from which disks are moved; Auxiliary $\rightarrow$ the temporary rod; Destination $\rightarrow$ the rod to which all disks must finally be moved.

Recursion trace for $n = 3$ (A = Source, B = Destination, C = Auxiliary)

```
T(3, A, B, C) | +-- T(2, A, C, B) | | | +-- T(1, A, B, C) -> Move disk 1 from A to C  
| +-- Move disk 2 from A to B | +-- T(1, C, A, B) -> Move disk 1 from C to B | +--  
Move disk 3 from A to C | +-- T(2, B, A, C) | +-- T(1, B, C, A) -> Move disk 1 from  
B to A +-- Move disk 2 from B to C +-- T(1, A, B, C) -> Move disk 1 from A to C
```

Recurrence relation:

$$T(n) = 2T(n - 1) + 1$$

Solving the recurrence, $T(n) = 2^n - 1$. For $n = 3$, $T(3) = 2^3 - 1 = 8 - 1 = 7$ moves.

Complexity: Time complexity is $O(2^n)$ (exponential), and space complexity is $O(n)$ because the recursion stack stores at most n activation records.

Q2. Explain performance analysis of algorithms. Discuss time complexity and space complexity with suitable algorithm examples.

The performance of a program is the amount of computer memory and time needed to run a program. We use two approaches to determine the performance of a program. One is analytical, and the other experimental.

In performance analysis, we use analytical methods, while in performance measurement, we conduct experiments.

Performance analysis of an algorithm can be measured in two ways

i) By calculating its time complexity

ii) By calculating its space complexity

Time Complexity: The time needed by an algorithm, expressed as a function of the size of the problem, is called the time complexity of the algorithm. It is the amount of computer time a program needs to run to completion. Time is usually counted in terms of the number of basic operations (comparisons, additions, assignments) executed.

Space Complexity: The total amount of memory required by an algorithm during its execution is its space complexity. The memory is needed for variables, constants, arrays, the recursion stack and dynamic memory allocation. It is expressed by the formula:

$$S(P) = C + S_p$$

- **Fixed part C** — independent of the input size; it includes program instructions, constants, simple variables and fixed-size data structures. This part never changes regardless of the input.
- **Variable part S_p** — depends on the size of the input; it includes arrays, dynamic memory, recursion stack and variable-size data structures. As the input grows, this part also grows.

Example — Algorithm SUM(A, n) that adds n numbers of an array:

```
Algorithm SUM(A, n) { sum := 0 for i := 1 to n do sum := sum + A[i] return sum }
```

Time complexity analysis:

Statement	Cost	Repetitions	Total
sum := 0	1	1	1
for i := 1 to n (condition checked)	1	$n + 1$	$n + 1$
sum := sum + A[i]	2	n	$2n$
return sum	1	1	1

$T(n) = 1 + (n + 1) + 2n + 1 = 3n + 3$. Ignoring the constant factor and lower-order terms, $T(n) = O(n)$.

Space complexity analysis:

- Array $A \rightarrow n$ words
- $n \rightarrow 1$ word
- $i \rightarrow 1$ word
- sum $\rightarrow 1$ word

Total space $S(P) = n + 3$. Hence $S(P) = O(n)$.

Q3. What is Binary Search? Explain the algorithm with a suitable example and analyze its time complexity.

Binary Search is an efficient searching technique used to locate an element in a **sorted** array. It works by repeatedly dividing the search space in half. In each step the middle element of the current search interval is compared with the key: if they are equal the search ends successfully; if the key is smaller the search continues in the left half; otherwise it continues in the right half. Each comparison therefore eliminates half of the remaining elements.

Algorithm

```
Algorithm BinarySearch(A, n, key) { low := 0 high := n - 1 while (low <= high)
do { mid := (low + high) / 2 if (A[mid] = key) then return mid if (key <
A[mid]) then high := mid - 1 else low := mid + 1 } return -1 }
```

Example / tracing — Array $A = \{10, 20, 30, 40, 50, 60, 70\}$, key = 50

Step	low	high	mid	A[mid]	Comparison	Action
1	0	6	3	40	$50 > 40$	Search right half: low = 4
2	4	6	5	60	$50 < 60$	Search left half: high = 4
3	4	4	4	50	$50 = 50$	Key found — return 4

The element 50 is found at **index 4** after only **3 iterations**.

Time complexity analysis

- **Best case:** the key is at the middle position and is found in the very first comparison $\rightarrow O(1)$.
- **Average case:** on average the search space halves every step $\rightarrow O(\log n)$.
- **Worst case:** the key is at one extreme of the array or absent; the search space is halved until it becomes empty $\rightarrow O(\log n)$.

Recurrence relation: $T(n) = T(n/2) + 1$, which gives $T(n) = O(\log n)$. The space complexity is $O(1)$ for the iterative version and $O(\log n)$ for the recursive version.

Q4. Explain Strassen's Matrix Multiplication algorithm with an Example.

Strassen's Matrix Multiplication is a Divide and Conquer algorithm, proposed by Volker Strassen, that multiplies two $n \times n$ matrices faster than the conventional method. Ordinary matrix multiplication of two $n \times n$ matrices requires $O(n^3)$ time. Strassen's algorithm reduces this to $O(n^{\log_2 7}) = O(n^{2.807})$, which is significantly faster for large matrices.

Divide and Conquer strategy

Given two matrices A and B of order $n \times n$, divide each matrix into four submatrices of size $(n/2) \times (n/2)$:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

The resultant matrix is:

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$C_{11} = A_{11}B_{11} + A_{12}B_{21} \quad C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

Conventional method — each block needs two matrix multiplications:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21} \quad C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21} \quad C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

This requires **8 multiplications** of submatrices. Strassen's improvement computes only **7 multiplications** by cleverly sharing the intermediate products.

Step 1 — Compute the seven products

$$P = (A_{11} + A_{22}) * (B_{11} + B_{22}) \quad Q = (A_{21} + A_{22}) * B_{11} \quad R = A_{11} * (B_{12} - B_{22}) \quad S = A_{22} * (B_{21} - B_{11})$$

$$T = (A_{11} + A_{12}) * B_{22} \quad U = (A_{21} - A_{11}) * (B_{11} + B_{12}) \quad V = (A_{12} - A_{22}) * (B_{21} + B_{22})$$

Step 2 — Compute the result matrix using only additions

$$C_{11} = P + S - T + V \quad C_{12} = R + T \quad C_{21} = Q + S \quad C_{22} = P + R - Q + U$$

Worked example — Multiply $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$.

Step 1 — Identify the elements: $A_{11}=1, A_{12}=2, A_{21}=3, A_{22}=4; B_{11}=5, B_{12}=6, B_{21}=7, B_{22}=8$.

Step 2 — Calculate the seven products:

$$P = (1 + 4) * (5 + 8) = 5 * 13 = 65 \quad Q = (3 + 4) * 5 = 7 * 5 = 35 \quad R = 1 * (6 - 8) = 1 * (-2) = -2$$

$$S = 4 * (7 - 5) = 4 * 2 = 8 \quad T = (1 + 2) * 8 = 3 * 8 = 24 \quad U = (3 - 1) * (5 + 6) = 2 * 11 = 22 \quad V = (2 - 4) * (7 + 8) = (-2) * 15 = -30$$

Step 3 — Compute the result matrix:

$$C_{11} = P + S - T + V = 65 + 8 - 24 - 30 = 19 \quad C_{12} = R + T = -2 + 24 = 22 \quad C_{21} = Q + S = 35 + 8 = 43 \quad C_{22} = P + R - Q + U = 65 - 35 - 2 + 22 = 50$$

Final result: $C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$ (verify: $1 \times 5 + 2 \times 7 = 19, 1 \times 6 + 2 \times 8 = 22, 3 \times 5 + 4 \times 7 = 43, 3 \times 6 + 4 \times 8 = 50$ ✓).

Recurrence relation:

$$T(n) = 7T(n/2) + O(n^2)$$

Using the Master's theorem, this gives $T(n) = O(n^{\log_2 7}) \approx O(n^{2.807})$, which is better than $O(n^3)$ for large matrices.

Q5. Explain the Divide and Conquer strategy with an algorithm and discuss its applications.

Divide and Conquer is an algorithm design technique in which a problem is repeatedly divided into smaller subproblems of the same type, each subproblem is solved independently, and their solutions are combined to obtain the final solution. It is widely used to design efficient algorithms for sorting, searching, matrix multiplication and many other problems.

Basic steps

- **Divide:** break the given problem into smaller subproblems.

- **Conquer:** solve each subproblem recursively; if the subproblem is small enough, solve it directly.
- **Combine:** merge the solutions of the subproblems to obtain the solution of the original problem.

Control abstraction (general algorithm)

```
Algorithm DANDC(P) { if SMALL(P) then return S(P) else { divide P into smaller
instances p1, p2, ..., pk, k >= 1 apply DANDC to each of these subproblems
return COMBINE(DANDC(p1), DANDC(p2), ..., DANDC(pk)) } }
```

Here **SMALL(P)** is a Boolean-valued function that decides whether the input size is small enough for the answer to be computed directly without further splitting; if so, function **S** is invoked. Otherwise the problem **P** is split into smaller instances **p1, p2, ..., pk** which are solved by recursive application of **DANDC**, and their solutions are combined by function **COMBINE**.

Recurrence relation — the running time of a Divide and Conquer algorithm is generally expressed as:

$$T(n) = aT(n/b) + f(n)$$

- **a** = number of subproblems
- **n/b** = size of each subproblem
- **f(n)** = time required to divide the problem and combine the solutions

Applications of Divide and Conquer

- **Binary Search** — searching in a sorted array, $O(\log n)$.
- **Merge Sort** — stable sorting, $O(n \log n)$.
- **Quick Sort** — in-place sorting, $O(n \log n)$ average case.
- **Strassen's Matrix Multiplication** — $O(n^{2.807})$.
- Other applications: finding maximum/minimum, large integer multiplication, exponentiation, and closest-pair problems.

Q6. Explain Merge Sort algorithm with an example and analyze its time complexity.

Merge Sort is a Divide and Conquer sorting algorithm that divides the array into smaller parts, sorts them, and then merges them back together in sorted order. The array is repeatedly split into two halves until each sub-array contains a single element (which is trivially sorted); then the sub-arrays are merged pairwise, always placing the smallest remaining value first.

Working

- **Divide:** recursively divide the list/array into two halves until it can no longer be divided (single-element subarrays).
- **Conquer:** each subarray is sorted individually using Merge Sort (a single element is already sorted).
- **Merge:** merge the sorted subarrays back together in sorted order, until the whole array is one sorted list.

Algorithm

```

Algorithm MergeSort(A, low, high) { if (low < high) then { mid := (low + high)
/ 2 MergeSort(A, low, mid) MergeSort(A, mid + 1, high) Merge(A, low, mid, high)
} }

```

Merge procedure

```

Algorithm MERGE(A, low, mid, high) 1. Compare the elements of the left and
right subarrays. 2. Copy the smaller element into a temporary array. 3. Repeat
until one subarray becomes empty. 4. Copy the remaining elements of the other
subarray. 5. Copy the temporary array back to A[low ... high].

```

Tracing — Input: A = [70, 30, 50, 10]

Divide phase

```

[70 30 50 10] | -----+----- | | [70 30] [50 10] | | ---+--- ---+--- | | | |
[70] [30] [50] [10]

```

Merge phase

- Merge [70] and [30] → [30, 70]
- Merge [50] and [10] → [10, 50]
- Merge [30, 70] and [10, 50]:

```

Compare 30 and 10 -> take 10 Compare 30 and 50 -> take 30 Compare 70 and 50 -> take
50 Remaining element -> 70

```

Final sorted array: **[10, 30, 50, 70]**

Time complexity analysis — recurrence relation:

$$T(n) = 2T(n/2) + n$$

Case	Time Complexity
Best case	$O(n \log n)$
Average case	$O(n \log n)$
Worst case	$O(n \log n)$

In every case the array is divided into two halves ($2T(n/2)$) and merged in linear time (n). By the Master's theorem this solves to **$O(n \log n)$** for all cases. The **space complexity** is **$O(n)$** because the merge step uses a temporary array of size n .

MODULE II — DISJOINT SETS, BACKTRACKING

Short Answer Questions

Q1. What is Hamiltonian cycle?

A **Hamiltonian cycle** is a cycle in a graph that visits every vertex exactly once and returns to the starting vertex. Each vertex appears in the cycle exactly once except the start vertex, which is also the end vertex. Example: in a graph with vertices A, B, C and D, the path $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ is a Hamiltonian cycle if each vertex is visited exactly once before returning to A. Not every graph contains a Hamiltonian cycle.

Q2. Write an Algorithm for Simple Union Operation?

The **Union** operation combines two disjoint sets into a single set. If i and j are the roots (representatives) of two different sets, making i the parent of j merges the two sets into one.

```
Algorithm UNION( $i, j$ ) { parent[ $j$ ] :=  $i$  }
```

Example: if Set 1 = {1, 2} with root 1 and Set 2 = {3, 4} with root 3, then UNION(1, 3) makes 1 the parent of 3, and the sets are merged into {1, 2, 3, 4}. The time complexity of simple union is **$O(1)$** .

Q3. What is the state-space tree in backtracking?

A **state-space tree** is a graphical (tree) representation of all possible states — that is, all candidate solutions and partial solutions — that are explored while solving a problem using backtracking. The root represents the initial state, each internal node represents a partial solution, and each leaf node represents a complete solution. Branches that cannot lead to a valid solution are **pruned** (discarded) as soon as they are found to be non-promising, which is what makes backtracking more efficient than exhaustive search.

Q4. Define the Find operation in a disjoint-set data structure.

The **Find** operation determines the representative (root) of the set containing a given element. Starting from the given element, it follows the parent links upwards through the tree until it reaches a node whose parent is -1 (the root), and returns that root. For example, if $S1 = \{1, 7, 8, 9\}$ then Find(7) = 1. In the simple implementation the worst-case time is **$O(n)$** .

Q5. How is a Heap Implemented?

A heap is implemented using an **array representation of a complete binary tree**. The elements are stored level by level in the array, and the parent and child nodes are accessed using index calculations. For an element at index i : its left child is at index $2i$ (or $2i+1$ in 0-based indexing), its right child at $2i+1$ (or $2i+2$), and its parent at index $i/2$ (or $(i-1)/2$). Because the tree is complete, no pointers are needed, and insertion and deletion are both efficient, taking $O(\log n)$ time.

Q6. What is the Sum of Subsets Problem?

The **Sum of Subsets** problem is to find all subsets of a given set of positive integers whose elements add up to a specified target sum. For example, for $S = \{10, 20, 30, 40\}$ and target 50, the solutions are $\{10, 40\}$ and $\{20, 30\}$. It is commonly solved using **backtracking**, where at each step an element is either included or excluded, and branches whose partial sum already exceeds the target are pruned to reduce the search space.

Long Answer Questions

Q1. Explain Disjoint Set Operations. Describe Union and Find algorithms with examples.

Two sets are **disjoint** if they do not have any common element. For example, $S_1 = \{1, 7, 8, 9\}$ and $S_2 = \{2, 5, 10\}$ are disjoint sets. The two fundamental operations on a collection of disjoint sets are **Union** and **Find**.

Disjoint set Union: if S_i and S_j are two disjoint sets, then their union $S_i \cup S_j$ consists of all elements x such that x is in S_i or S_j . Example: $S_1 = \{1, 7, 8, 9\}$ and $S_2 = \{2, 5, 10\}$ give $S_1 \cup S_2 = \{1, 2, 5, 7, 8, 9, 10\}$.

Find: the Find operation finds the root node (representative element) of the set containing a given element. For the sets $S_1 = \{1, 7, 8, 9\}$, $S_2 = \{2, 5, 10\}$, $S_3 = \{3, 4, 6\}$: $\text{Find}(4) = S_3$, $\text{Find}(5) = S_2$, $\text{Find}(7) = S_1$.

Simple Union

The Simple Union operation combines two disjoint sets into one by making one root the parent of the other root.

- Consider all elements as individual sets.
- Initialize the parent array with -1 (each element is its own root).
- Select two root nodes.
- Make one root the parent of the other.
- Update the parent array after every union.

```
Algorithm SimpleUnion(i, j) { P[j] := i }
```

where i is the root of the first set and j is the root of the second set. After the operation, i becomes the parent of j .

Example — elements 1, 2, 3, 4, 5, 6

- Initial state: $P = [-1, -1, -1, -1, -1, -1]$ — every element is its own set.
- $\text{Union}(1, 2)$: $P[2] = 1 \rightarrow$ tree: 1 is parent of 2.
- $\text{Union}(3, 4)$: $P[4] = 3 \rightarrow$ tree: 3 is parent of 4.
- $\text{Union}(5, 6)$: $P[6] = 5 \rightarrow$ tree: 5 is parent of 6.
- $\text{Union}(1, 3)$: $P[3] = 1 \rightarrow$ tree: 1 becomes parent of 3 (and grandparent of 4).
- $\text{Union}(1, 5)$: $P[5] = 1 \rightarrow$ tree: 1 becomes parent of 5 (and grandparent of 6).

Final tree — all elements are in one set rooted at 1:

```
1 / | \ 2 3 5 | | 4 6
```

Final parent array:

i	1	2	3	4	5	6
P[i]	-1	1	1	3	1	5

Simple Find

The Simple Find operation finds the root node (representative) of the set containing a given element.

```
Algorithm Find(i) { while (P[i] > 0) do i := P[i] return i }
```

- Start from the given node.
- Follow the parent links upward.
- Continue until a node with parent -1 is reached.
- That node is the root — return it.

Example — Find(4):

```
Step 1: i = 4, P[4] = 3 -> move to parent, i = 3 Step 2: i = 3, P[3] = 1 -> move to parent, i = 1 Step 3: i = 1, P[1] = -1 -> root found, return 1
```

Therefore Find(4) = 1.

Time complexity: Simple Union is **O(1)**; Simple Find is **O(n)** in the worst case (when the tree is a long chain). With heuristics such as union by rank and path compression, both operations can be made almost O(1) — the amortized inverse-Ackermann $O(\alpha(n))$.

Q2. Explain the N-Queens Problem using Backtracking. Illustrate the solution for a 4-Queens problem.

The **N-Queens problem** is a classic backtracking problem that involves placing N queens on an N×N chessboard such that no two queens attack each other. A queen can move horizontally, vertically and diagonally, so the placement must satisfy three conditions:

- No two queens are placed in the same row.
- No two queens are placed in the same column.
- No two queens are placed on the same diagonal.

The problem is solved using **backtracking**: queens are placed one row at a time. For each row, every column is tried; a queen is placed only if it does not attack any previously placed queen. If a queen cannot be placed safely in the current row, the algorithm **backtracks** to the previous row and moves that queen to the next valid column. This continues until all queens are placed successfully or all possibilities are exhausted.

Illustration — the 4-Queens problem

Q1 is placed in the first row, first column. The second queen must be in row 2, and cannot be in column 1 (same column) or column 2 (same diagonal). So Q2 is placed in column 3.

```
Row 1: [Q1] [ ] [ ] [ ] Row 2: [ X ] [ X ] [Q2] [ ] (col1 = same column, col2 = same diagonal)
```

Now Q3 cannot be placed anywhere in row 3 without attacking Q1 or Q2. So the algorithm **backtracks**: it removes Q3, moves Q2 to the next column (column 4), and tries again. Still no position works for Q3, so it backtracks further: Q2 is removed and Q1 is moved to column 2.

```
Row 1: [ ] [Q1] [ ] [ ] Row 2: [ ] [ ] [ ] [Q2] (col4 – no diagonal clash with Q1)
Row 3: [Q3] [ ] [ ] [ ] (col1 – safe) Row 4: [ ] [ ] [Q4] [ ] (col3 – safe)
```

Hence the solution to the 4-Queens problem is $x_1 = 2, x_2 = 4, x_3 = 1, x_4 = 3$ — the first queen is placed in column 2, the second in column 4, the third in column 1 and the fourth in column 3.

Complexity: the worst-case time complexity of the N-Queens problem is $O(N!)$ (the first queen has N choices, the second N–2, etc.), and the space complexity is $O(N)$ for the recursion stack and the column array.

Q3. Describe the general method of Backtracking with an algorithm and write its applications.

Backtracking is a problem-solving approach that systematically tries out all possible candidate solutions and selects the desired ones. It is generally used for problems that have multiple solutions, such as decision and optimization problems. Instead of generating the complete search space, backtracking builds a solution **one component at a time** and abandons (prunes) a partial solution as soon as it can be shown that it cannot be extended to a valid full solution.

Terminology

- **Root node** — represents the initial state of the problem.
- **Live node** — a node that has been generated but not yet explored.
- **E-node (expansion node)** — the live node that is currently being expanded.
- **Dead node** — a node that cannot lead to a solution and is discarded.
- **Answer node** — a node that represents a complete valid solution.

State-space tree

A state-space tree is a graphical representation of all possible states generated during the search. The root is the initial state, internal nodes are partial solutions, and leaf nodes are complete solutions. Invalid branches are pruned:

```
Root / | \ A B C / \ / \ D E F G Root -> initial state Internal nodes -> partial
solutions Leaf nodes -> complete solutions
```

General backtracking algorithm

```
Algorithm Backtrack(k) { if a solution is found then print the solution else {
for each candidate x do { if x is promising then { include x Backtrack(k + 1)
remove x } } } }
```

Steps in backtracking

- Start from the root node.
- Generate a candidate solution.
- Check whether it is promising.
- If promising, continue the search deeper.
- If not promising, backtrack to the previous step.
- Repeat until all solutions have been explored.

Applications of backtracking

- N-Queens Problem

- Graph Coloring (m-coloring)
- Hamiltonian Cycle
- Sum of Subsets
- Other applications: the knapsack problem, Sudoku, crossword puzzles, and solving mazes.

Q4. Generate the state space tree and find the solutions for the subset sum for $N = 4$, $M = 50$, $W = \{10, 20, 30, 40\}$ using backtracking approach.

Problem: given a set of positive integers $S = \{10, 20, 30, 40\}$ and a target sum $d = 50$, find all subsets of S whose sum is exactly 50. Backtracking is used so that non-promising subsets are discarded as early as possible, reducing the search space.

Let x_1, x_2, x_3, x_4 represent the elements 10, 20, 30, 40 respectively. In the state-space tree, a **1** means the element is included in the subset and a **0** means it is excluded.

Algorithm

```
Algorithm SumOfSubsets(i, sum) { if sum = d then { print the subset return } if
sum > d or i = n then return // include S[i] SumOfSubsets(i + 1, sum + S[i]) //
exclude S[i] SumOfSubsets(i + 1, sum) }
```

Here i is the current element index, sum is the sum of elements selected so far, and d is the target sum.

Search table

Step	Current subset	Sum	Action
1	{}	0	Start
2	{10}	10	Add next
3	{10, 20}	30	Add next
4	{10, 20, 30}	60	Backtrack (sum > 50)
5	{10, 20, 40}	70	Backtrack (sum > 50)
6	{10, 30}	40	Add next
7	{10, 30, 40}	80	Backtrack (sum > 50)
8	{10, 40}	50	Solution ✓
9	{20}	20	Continue
10	{20, 30}	50	Solution ✓
11	{20, 40}	60	Backtrack (sum > 50)
12	{30}	30	Continue
13	{30, 40}	70	Backtrack (sum > 50)
14	{40}	40	Continue
15	{ } (all excluded)	0	Done

State-space tree (partial — promising branches only)

```
(0, { }) / \ (10, {10}) (0, { }) / \ ... (30, {10, 20}) (40, {10, 30}) / \ / \
(60, {10, 20, 30}) (70, {10, 20, 40}) (50, {10, 30, 40}) (50, {10, 40}) prune: sum>50 prune:
sum>50 prune: sum>50 SOLUTION ... (20, {20}) / \ (50, {20, 30}) (60, {20, 40}) SOLUTION
prune: sum>50
```

Solutions: the subsets that sum to 50 are **{10, 40}** and **{20, 30}**.

Q5. Explain the Graph Coloring problem and solve it using the Backtracking technique.

Graph coloring (m-colorability): Let G be a graph and m a given positive integer. The m -colorability decision problem asks whether the nodes of G can be colored using at most m colors so that no two adjacent nodes have the same color. If d is the maximum degree of the graph, the graph can always be colored with $d + 1$ colors. The smallest value of m for which a valid coloring exists is called the **chromatic number**.

Backtracking approach: vertices are colored one by one. For each vertex, the smallest color that does not clash with any of its already-colored neighbours is tried. If a vertex cannot be given any valid color, the algorithm backtracks to the previous vertex and tries the next color for it. This continues until all vertices are colored or all possibilities are exhausted.

General algorithm

```
Algorithm mColoring(k) { while (true) do { NextValue(k) if x[k] = 0 then return
if k = n then print(x[1 ... n]) else mColoring(k + 1) } }
```

```
Algorithm NextValue(k) { repeat { x[k] := (x[k] + 1) mod (m + 1) // next color
if x[k] = 0 then return // no color available for j := 1 to n do if adjacent(k,
j) and x[j] = x[k] then break // clash -> try next color if j = n + 1 then
return // color is valid } }
```

Example — a cycle of four vertices

Vertices A, B, C, D with edges A–B, B–C, C–D, D–A (adjacency matrix shown below). Colors: 1 = Red, 2 = Green, 3 = Blue. Initially $x = [0, 0, 0, 0]$.

	V1 (A)	V2 (B)	V3 (C)	V4 (D)
V1 (A)	0	1	0	1
V2 (B)	1	0	1	0
V3 (C)	0	1	0	1
V4 (D)	1	0	1	0

Coloring process

- **Step 1:** assign V1 (A) = color 1 (Red) $\rightarrow x = [1, 0, 0, 0]$.
- **Step 2:** V2 (B) is adjacent to V1, so color 1 is rejected; assign color 2 (Green) $\rightarrow x = [1, 2, 0, 0]$.
- **Step 3:** V3 (C) is adjacent only to V2; color 1 (Red) is valid $\rightarrow x = [1, 2, 1, 0]$.
- **Step 4:** V4 (D) is adjacent to V1 and V3; colors 1 is rejected, color 2 (Green) is valid $\rightarrow x = [1, 2, 1, 2]$.

Final coloring

Vertex	Color number	Color name
V1 (A)	1	Red
V2 (B)	2	Green
V3 (C)	1	Red
V4 (D)	2	Green

The 4-vertex cycle is colored with just 2 colors (its chromatic number is 2, since it is bipartite — an even cycle).

Complexity: worst-case time is $O(m^n)$ (n = number of vertices, m = number of colors), since in the worst case every one of the m colors may be tried for each of the n vertices. Space complexity is $O(n)$ for the recursion stack and the color array.

Q6. What is a Hamiltonian Cycle? Explain the Backtracking solution for finding Hamiltonian Cycles in a graph.

A **Hamiltonian cycle** is a cycle in a graph that visits every vertex exactly once and returns to the starting vertex. Each vertex is visited only once, the first and last vertices are the same, and not every graph contains a Hamiltonian cycle. The problem of deciding whether such a cycle exists is NP-complete and is typically solved with backtracking.

Idea of the backtracking solution

- Start from the first vertex.
- Add one vertex at a time to the path.
- Before adding a vertex, check whether it is valid.
- If a wrong choice is made, backtrack and try another vertex.
- Continue until all vertices are included and the last vertex connects back to the first.

Conditions to add a vertex

- It is connected to the previous vertex in the path.
- It has not been used before.
- If it is the last vertex, it must connect back to the starting vertex.

Algorithm

```
Algorithm Hamiltonian(k) { if all vertices are included then print the cycle
else { for every vertex v do { if IsValid(v) then { add v to the path
Hamiltonian(k + 1) remove v (backtrack) } } } }
```

```
Algorithm IsValid(v) { 1. Check whether v is connected to the previous vertex.
2. Check whether v is already used. 3. If v is the last vertex, check whether
it connects back to the first vertex. }
```

Step-by-step example — graph with vertices 1–5 connected as shown (1–2, 1–3, 2–3, 2–5, 5–4, 3–4):

- **Step 1:** start from vertex 1 → path = [1]
- **Step 2:** choose vertex 2 → path = [1, 2]

- **Step 3:** choose vertex 5 → path = [1, 2, 5]
- **Step 4:** choose vertex 4 → path = [1, 2, 5, 4]
- **Step 5:** choose vertex 3 → path = [1, 2, 5, 4, 3]
- **Step 6:** check whether vertex 3 connects back to vertex 1 — yes, so the Hamiltonian cycle is **1 → 2 → 5 → 4 → 3 → 1**.

State-space tree

```
1 / \ 2 3 | 5 | 4 | 3 | Hamiltonian Cycle (3 connects back to 1)
```

Complexity: worst-case time complexity is **$O(n!)$** because, in the worst case, all permutations of the n vertices are examined; space complexity is **$O(n)$** for the recursion stack and the path array.